

MEDIATEK

MT8676 Yocto Secure Boot User Manual

Version: 1.0
Publication date: 2024-11-20

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-11-20	Steven.Gao	Official release

Table of Contents

Version History 2

Table of Contents 3

List of Figures 3

List of Tables 3

1 Secure Boot 4

1.1 Overview 4

1.1.1 Abbreviations 4

1.1.2 BootROM 4

1.2 Secure Boot 5

1.2.1 Secure Boot Check Process 5

1.3 Download Agent Authentication (DAA) 6

1.4 Security Feature Configuration 7

1.4.1 Generate a Key Pair 7

1.4.2 Enable Secure Boot 7

1.4.3 Compilation Software 8

1.4.4 Independent Remote Signature Environment 8

1.5 Signature DA 9

1.5.1 Generate dakey.h 9

1.5.2 Signature DA 10

1.5.3 Generate Authfile 10

Exhibit 1 Terms and Conditions 11

List of Figures

Figure 1-1. Secure boot check process 6

List of Tables

Table 1-1. Abbreviations 4

1 Secure Boot

1.1 Overview

This document aims to provide an overview of the secure boot feature in the MT8676 SoC and its accompanying SDK. During the design and deployment cycle of this product, functionalities and features may change; these changes will be documented in subsequent versions of this document. Additionally, this document describes how to enable secure features.

1.1.1 Abbreviations

Table 1-1. Abbreviations

Abbreviation	Explanation
DA	Download Agent (including DA_BR and DA_PL)
DAA	Download Agent Authentication
DRAM	Data Random Access Memory
eFuse	Electronic Fuse
eMMC	Embedded MultiMedia Card
EVB	Evaluation Board
LK	Little Kernel
NVM	Non-Volatile Memory
PC	Power Control
PLL	Phase-Locked Loop
ROM	Read-only Memory
SBC	Secure Boot Check
SDK	Software Development Package
SoC	System-on-Chip
SRAM	Static Random Access Memory
TEE	Trusted Execution Environment
UART	Universal Asynchronous Receiver/Transmitter
USB	Universal Serial Bus
USBDL	USB Download

1.1.2 BootROM

BROM (BootROM) is the software in SoC ROM that cannot be modified. It is the first software executed by the application processor. The main work summary is as follows:

- Perform basic hardware configuration for the System on Chip (SoC), such as setting the Phase-Locked Loop (PLL) and clock, to initiate system startup.

- Load the first-stage bootloader from the startup device UFS/eMMC/NAND to initiate the system.
- Communicate with the host PC tool to load Download Agent (DA) software for mirroring downloads to external Flash devices that lack software or have corrupted software.
- Perform secure boot checks on the first-stage bootloader or download agent using multiple secure boot check keys.

1.2 Secure Boot

1.2.1 Secure Boot Check Process

Secure boot check process for establishing a trusted execution chain from hardware trust. It can be enabled by burning SBC_EN in eFuse. This process is performed every time the system recovers from power-on reset. Figure 1-1 shows the basic secure boot check flow. Detailed description is as follows:

1. After power-on reset, BROM uses the following process to verify the secure boot check (SBC) public key (SBC_PUBK) in non-volatile memory (NVM, such as eMMC or NAND). BROM sequentially uses the public key hash (SBC_PUBK_HASH) from eFuse.
 - (1) Read the hash value(SBC_PUBK_HASH) of the secure boot check public key (SBC_PUBK) from eFuse.
 - (2) Read the SBC public key (SBC_PUBK) from NVM.
 - (3) Calculate the hash value of the data from (2).
 - (4) Check whether the data from (1) and (3) is the same.
2. BROM loads and verifies the First-loader, which is the first stage bootloader. First, BROM reads the First-loader from NVM into the SoC's SRAM and uses the Secure Boot Check Public Key (SBC_PUBK) to authenticate the First-loader. If the First-loader verification is successful, the First-loader will be executed. The verification method uses SHA256 to compute the hash value and RSA (2048 bits) to verify the signature. The First-loader contains key cert, content cert, and image content; the specific process is as follows:
 - (1) Use SBC_PUBK to verify First-loader key cert.
 - (2) If the First-loader key cert validation is successful, compare the image pub key in the key cert with the image pub key in the content cert.
 - (3) If (2) is consistent, then use the image pub key to verify the First-loader content cert.
 - (4) Verification successful, and finally calculate the hash value of the First-loader image and compare it with the hash value stored in the content cert.

Note:

- If the external storage is NAND flash memory, BROM supports the second copy of First-loader. When BROM cannot load the first copy of First-loader, BROM will attempt to load/authenticate the second copy of First-loader.
3. After the first stage bootloader, fitimage was introduced for subsequent verification. This includes images such as kernel and TEE.
 4. First-loader loads the Linux kernel loading program from NVM into a non-secure DRAM area and verifies it. It uses the VERIFIED public key embedded in First-loader to authenticate the kernel FIT signature and detects the HASH values of each sub-image in the Fit image. The verification method uses SHA256 to compute the hash value, RSA (2048 bits), and MTK or PSS padding for verification.

5. First-loader loads the TEE loader from NVM into a non-secure DRAM area and verifies it. It uses the VERIFIED public key embedded in the First-loader to authenticate and verify the TEE FIT signature and detects the HASH values of each sub-image in the Fit image. The verification method uses SHA256 to compute the hash value, RSA (2048 bits), and MTK or PSS padding for verification.
6. After executing in the Linux system, the kernel can implement sampling verification of the read-only system through the rootf_check function provided by mtk in the Linux kernel.

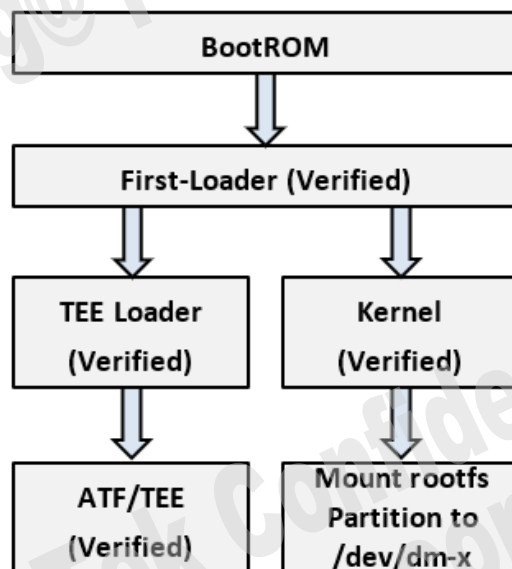


Figure 1-1. Secure boot check process

1.3 Download Agent Authentication (DAA)

When the device software in the external memory (eMMC/NAND/...) is empty or damaged, the host PC tool can communicate with the BROM in the SoC, load a DA software into the SRAM of the SoC and execute DA to download the image process. When Enable_DAA is burned, the DA will be authenticated by BROM, and the following process is called Download Agent Authentication (DAA).

1. The host PC tool sends the authentication file (AuthFile) to BROM via USB or UART.
2. BROM authenticates the AuthFile.
 - (1) BROM reads the Secure Boot Check (SBC) public key (SBC_PUBK) from the AuthFile and uses the public key hash (SBC_PUBK_HASH~SBC_PUBK_HASH1) and the corresponding disable bits (SBC_PUBK_HASH_DIS~SBC_PUBK_HASH1_DIS) to authenticate the hash of the SBC key.
 - (2) BROM uses SBC_PUBK to verify AuthFile.
 - (3) BROM obtains the DAA key from the AuthFile.
3. The host PC tool sends DA.
4. BROM uses the DAA key to verify DA through SHA256/RSA-2048.
5. BROM jumps to DA to execute the firmware download process.

Note:

- The SBC_PUBK and DAA keys can differ to enhance security levels.

1.4 Security Feature Configuration

1.4.1 Generate a Key Pair

Generate two pairs of keys (including private keys and public keys): 1. SBC_KEY 2. VERIFIED_KEY. Place them in the directory: meta/meta-MediaTek/conf/machine/keys.

1. The command to generate a private key:

```
openssl genrsa -F4 -out sbc_key.pem 2048
openssl genrsa -F4 -out verified_key.pem 2048
openssl req -batch -new -x509 -key verified_key.pem -out verified_key.crt
```

2. The command to generate a public key:

```
openssl rsa -in sbc_key.pem -pubout > sbc_pubk.pem
openssl rsa -in verified_key.pem -pubout > verified_pubk.pem
```

Generate the Digital Authentication (DA) key pair for signing and verification using the same method (da_prvk.pem/da_pubk.pem). The DA can be identical to or different from the Secure Boot Controller (SBC) key.

1.4.2 Enable Secure Boot

1.4.2.1 Generate SCB Public Key HASH

1. Tool path:

meta/meta-mediatek/recipes-bsp/lk/files/pbp

2. Command:

```
chmod 777 der_extractor
python pbp.py -j sbc_key.pem -func keyhash_pss -o keyhash
```

1.4.2.2 Blow SBC_PUBK0_HASH_Field

Step 1: Use `hexdump -C keyhash` or `xxd -c 32 keyhash` to display the hexadecimal keyhash.

Note:

- You must use the `-c` parameter to generate a standard hexadecimal ASCII display result.

```
hexdump -C keyhash
```

```
00000000 16 b1 oe fc 5e 4e 06 76 e9 d9 6e 40 0c 51 ca 36 | .....
00000010 d1 be 93 d2 67 fd 3e af db f6 f7 89 4a 2c 40 18 | ..... .
```

```
xxd -c 32 keyhash
```

```
00000000 16b1 oefc 5e4e 0676 e9d9 6e40 0c51 ca36 d1be 93d2 67fd 3eaf dbf6 f789 4a2c
4018      .....
```

Do not use `hexdump keyhash` **for display.**

```
hexdump keyhash
```

```
00000000 b116 fc0e 4e5e 7606 d9e9 406e 510c 36ca
00000010 bed1 d293 fd67 ad3e f6db 89f7 2c4a 1840
```

Step 2: Convert the result to a hexadecimal string. Remove the spaces between characters:

```
16b10efc5e4e0676e9d96e400c51ca36d1be93d267fd3eafdbf6f7894a2c4018
```

Step 3: Execute the ewriter command:

```
ewriter 1 0 32 16b10efc5e4e0676e9d96e400c51ca36d1be93d267fd3eafdbf6f7894a2c4018
```

For more detailed usage of the ewriter tool, refer to the *MTK_eFuse_Writer_User_Guide*.

Note:

- To enable the BROM secure boot verification function of the SoC, in addition to writing the above SBC_PUBK0_HASH field, it is also necessary to write SBC_EN. The eFuse can only be written once, so it is not recommended to write eFuse to enable the BROM secure boot verification function of the IC during the project development phase.

1.4.2.3 Enable Secure Boot Configuration in the Software

```
meta/meta-mediatek-mt8xxx/conf/machine/[project].conf
```

```
SECURE_BOOT_ENABLE = "yes"
ENABLE_ROOTFS_CHECK= "yes"
```

This configuration is enabled by default in the public version to allow software-level secure boot and secure download without using eFuse, facilitating the verification of the secure boot process for software above LK.

1.4.3 Compilation Software

Compile the entire project.

1.4.4 Independent Remote Signature Environment

The MTK SDK supports an independent signing environment. In this mode, only the public key can be placed in the code repository, with the final signing operation performed later.

1.4.4.1 Preparation of the Independent Signature Environment

Step 1: Enable the configuration for remote standalone signing. After enabling STANDALONE_SIGN_PREPARE, the signing action will be placed in a standalone signing environment; therefore, after enabling this macro, the compiled image is by default unsigned.

```
SECURE_BOOT_ENABLE = "yes"
STANDALONE_SIGN_PREPARE = "yes"
ENABLE_ROOTFS_CHECK = "yes"
```

Step 2: Keep the `verified_pubk.pem` file in the `meta/meta-MediaTek/conf/machine/keys` directory, and place other keys in a separate signing environment.

Step 3: Compile the image.

1.4.4.2 Signed Image

The independent signature environment toolkit is located at: `meta/meta-MediaTek-mt8676/recipes-devtools/standalone_sign_env`. Customers can place the toolkit on a specific server according to their needs.

Step 1:

Place the relevant keys `${SBC_KEY}.pem` and `${VERIFIED_KEY}.pem` into the 'keys' folder.

Step 2: Execute the signing script: `sh sign_images.sh`

If everything functions correctly, the signed image will be placed in the 'signed' folder.

Note:

- `sign_images.sh` is an example file of the MTK SDK. It is strongly recommended that you use this file and modify it according to your needs.

1.5 Signature DA

DA Authentication (DAA) is a solution from MediaTek that ensures the security of firmware download progress. Before starting authentication, ensure that Secure Boot Check (SBC) is enabled and that the secure boot test has passed. The following steps are required for signing DAA using Android-related scripts and source code in Yocto: 1. Enable SBC. 2. Verify secure boot test completion. 3. Utilize Android scripts and source code for DAA signing.

1.5.1 Generate dakey.h

Export the DA public key (`da_pubk.pem`), use `der_extractor` to generate `dakey.h`, this tool is located in the `android source vendor/MediaTek/proprietary/scripts/sign-image_v2/der_extractor/` directory, please place the generated `dakey.h` in the following yocto path:

`[LK2] $LK2/target/$PROJECT/include/dakey.h`

Command:

```
chmod 777 der_extractor
python pem_to_der.py da_pubk.pem da_pubk.der
./der_extractor da_pubk.der dakey.h ANDROID_SBC
```

1.5.2 Signature DA

1. Key path settings

Export the DA public key (da_prvk.pem) and place it in the Android path

vendor/mediatek/proprietary/scripts/secure_chip_tool/custom_keys directory.

2. Place the DA that needs to be signed in the prebuilt/resignda/ directory, and execute the following command to sign the DA:

```
cd vendor/mediatek/proprietary/scripts/secure_chip_tool/  
python MTK/resign_da.py prebuilt/resignda/DA_BR.bin MT6897  
settings/Legacy/da/bbchips_pss.ini all out/resignda/DA_BR-resing.bin
```

1.5.3 Generate Authfile

The public key of DA is included in the authfile. BROM uses it to verify DA. Therefore, if DAA is enabled, the authfile is required when using flashtool to download the image. All .ini files located in the following path (Android branch) have been configured for this project.

./vendor/mediatek/proprietary/scripts/secure_chip_tools/settings/Legacy/authfile/

Replace the .pem files located at the following positions: DA and root private key (rename sbc_key.pem to root_prvk.pem).

./vendor/mediatek/proprietary/scripts/secure_chip_tools/custom_keys/

Authfile generation command:

```
python MTK/toolauth.py -i settings/Legacy/authfile/toolauth_key.ini -g  
settings/Legacy/authfile/toolauth_gfh_config_pss.ini out/toolauth/auth_sv5.auth
```

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.